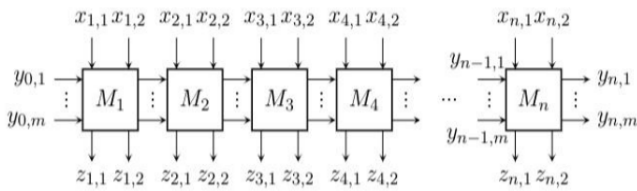


Reti iterative e sequenziali

venerdì 27 febbraio 2026 11:32

I processori moderni, implementano e gestiscono programmi fino a 512 bit. Quindi i circuiti combinatori visti finora non sono tanto "adatti". Quindi si cerca di "spezzettare il problema" in vari sotto problemi, i quali verranno ricomposti alla fine. Quindi si prendono i bit in ingresso, quelli in uscita e li studio in sotto blocchi. Attenzione all'interdipendenza tra i blocchi:



Dove :

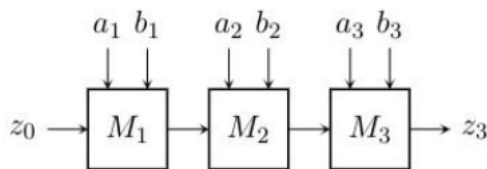
Vettore y : rappresenta le informazioni di stato trasferite da un modulo al successivo

- L'ultimo modulo può esporre parte di questa informazione all'esterno, ad esempio per notificare dettagli circa il risultato finale dell'operazione

Vettore x : rappresenta il dato in input, decomposto tra i vari moduli

Vettore z : rappresenta l'output, calcolato iterativamente dai moduli

Attenzione al tempo di propagazione : dipende dal numero dei blocchi . Vediamo ora delle applicazioni : **il comparatore** : modulo hardware che confronta due parole binarie : quindi prende in input due input e ci dice se questi due numeri sono uguali o no .



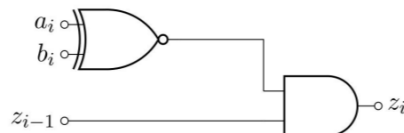
Da notare che z è un'informazione che viene propagata : nel nostro caso è il risultato del confronto. Quindi andiamo a costruire la tabella di verità :

z_{i-1}	a_i	b_i	z_i
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

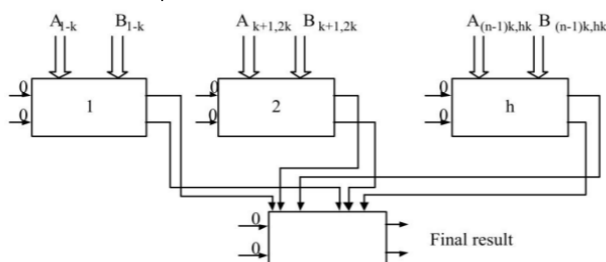
Se i bit precedenti sono uguali l'uscita vale 1 solo in due casi : devo propagare anche l'informazione che il modulo corrente sta osservando , solo se i due bit correnti sono 1 (carry in =1).

$$z_i = z_{i-1} \bar{a} \bar{b} + z_{i-1} a b = z_{i-1} (a \odot b)$$

Attenzione al primo modulo : ingressi devono essere configurato/forzato ad 1 , in quanto non ci è stato nessun confronto . Dal punto di vista circuitale :



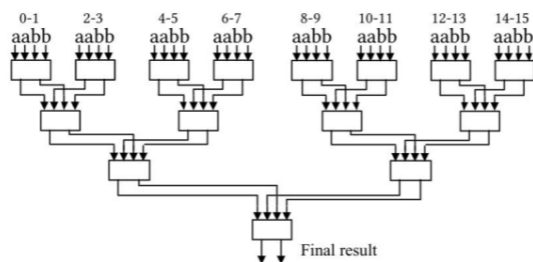
Quanto detto finora si ha solo uguaglianza , ma in generale si devono vedere anche se i numeri sono maggiori o minori . Di queste reti (o meglio quelle con n arbitrario di moduli) il problema è il tempo di propagazione , che si stabilizzerà dopo . Posso velocizzare?? Si potrebbe parallelizzare : parte dei calcoli svolti in parallelo : **comparatore veloce** : confronto i bit a coppia (h blocchi di k bit) ed ogni blocco è internamente un comparatore :



Il problema è il numero di input alla rete sottostante : è abbastanza elevato . Questa potrebbe essere una soluzione , ma visto che spesso si usano parole con lunghezza di potenze di 2 , potrei pensare ad una

logica ad albero : **comparatore ad albero** : divido il mio circuito , calcolando in parallelo il risultato ed aggregando il risultato in parallelo :

Ad esempio, per interi a 16 bit:



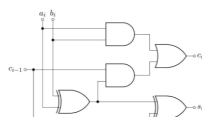
Oltre ai comparatori, ci servono anche i sommatore : circuiti elementari che permette di calcolare la somma ad un solo bit , inoltre genera un'informazione di riporto (carry-in) che non viene presa in considerazione dal modulo successivo : vediamo **l'half adder** :

$$s = a \oplus b \quad c = a \cdot b$$



Questo è un circuito molto veloce , ma non lo posso usare per fare il sommatore ad n bit, in quanto genero l'informazione di stato, ma non viene preso in considerazione. Vediamo ora quello completo : il **full adder (considera il riporto precedente)** :

$$s_i = c_{i-1} \oplus a_i \oplus b_i \quad c = a_i b_i + c_{i-1} (a_i \oplus b_i)$$



$$s_i = f_1(a_i, b_i, c_{i-1})$$

$a_i b_i$	00	01	11	10
0	0	1	0	1
1	1	0	1	0

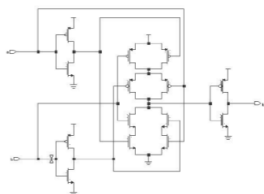
$$c = f_2(a_i, b_i, c_{i-1})$$

$a_i b_i$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$s_i = c_{i-1} \oplus a_i \oplus b_i$$

$$c = a_i b_i + c_{i-1} (a_i \oplus b_i)$$

Da notare che il riporto è più lento della somma perché aggiungo una porta and (prodotto): anche se ho un termine in comune , calcolo due pezzi di due funzioni diverse : risparmio componenti elettronici . In questo circuito si usa un doppio xor binario rispetto ad uno ternario per via del costo elettronico in componenti :



Inoltre il tempo di propagazione di questa porta è spaventoso ! Possiamo migliorare questo circuito ? Andiamo a riorganizzare la mia rete iterativa per andare a calcolare un qualche risultato in parallelo (il bit di carry) : cerco di sbirciare il carry per evitare perdite di tempo : **il carry lookahead adder (carry in parallelo alla somma)**:

$$s_i = c_{i-1} \oplus a_i \oplus b_i \text{ e } c = a_i b_i + c_{i-1} (a_i \oplus b_i)$$

$$\text{Poniamo: } G_i = a_i b_i \text{ e } P_i = a_i \oplus b_i$$

Possiamo riscrivere le equazioni come:

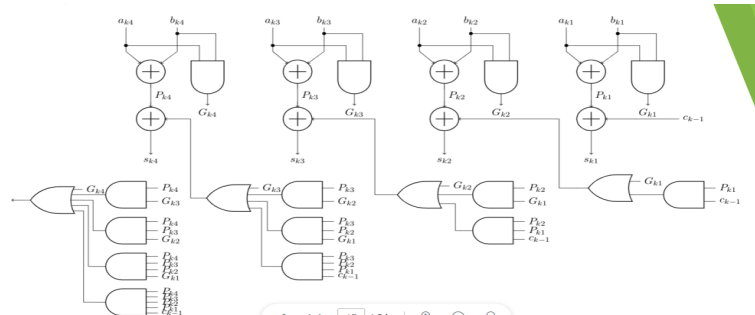
- $s_i = P_i \oplus c_{i-1}$
- $c = G_i + P_i c_{i-1}$

Dove Gi è il generatore di riporto , mentre Pi è il propagatore.

In questo circuito a posto di prendere coppie di bit , ne prendiamo 4 coppie di bit :

$$\begin{aligned} c_{k4} &= G_{k4} + P_{k4} c_{k3} = G_{k4} + P_{k4} (G_{k3} + P_{k3} c_{k2}) = \\ &= G_{k4} + P_{k4} (G_{k3} + P_{k3} (G_{k2} + P_{k2} c_{k1})) = \\ &= G_{k4} + P_{k4} (G_{k3} + P_{k3} (G_{k2} + P_{k2} (G_{k1} + P_{k1} c_{k-1}))) = \\ &= G_{k4} + P_{k4} G_{k3} + P_{k4} P_{k3} G_{k2} + P_{k4} P_{k3} P_{k2} G_{k1} + P_{k4} P_{k3} P_{k2} P_{k1} c_{k-1} \end{aligned}$$

Il circuito è :



Vediamo ora lo **shifter** : muove a dx o sx i bit di una parola binaria . Decido inoltre la cifra che faccio entrare : 0 oppure 1 . Vediamo u esempio :

0100-> shifto a sx di una posizione -> 1000

Shiftare a sx si moltiplica per 2^x; a destra invece divido per 2^x.

1010-> shifto a dx di una posizione -> 0101

Attenzione alla cifra che entra : se interi positivi entra uno 0 (shift logico) ,mentre se cp2 entra un 1 (shift aritmetico).

1010-> shifto a dx -> 1101

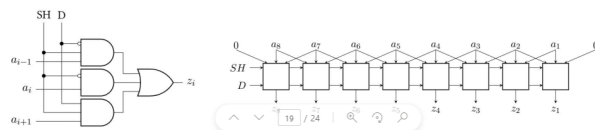
Il circuito è:

- Possiamo descrivere il circuito che calcola il valore del bit z_i in uscita con il seguente sistema:

$$z_i = \begin{cases} a_i & \text{se } SH = 0 \\ a_{i-1} \cdot \bar{D} + a_{i+1} \cdot D & \text{se } SH = 1 \end{cases}$$

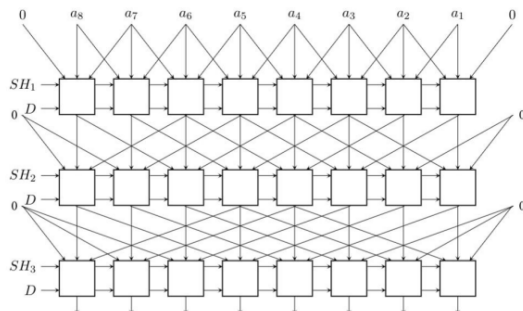
- Pertanto l'equazione che calcola il valore del bit diventa:

$$z_i = \overline{SH} \cdot a_i + SH \cdot (a_{i-1} \cdot \bar{D} + a_{i+1} \cdot D)$$

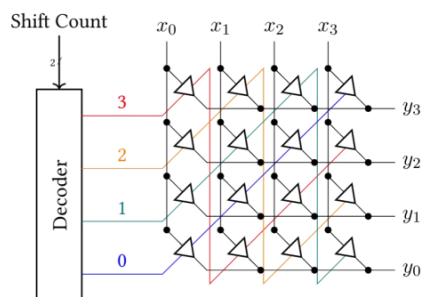


Se shift=0 non fai shift , d=0 trasla a dx ;d=1 trasla a sx

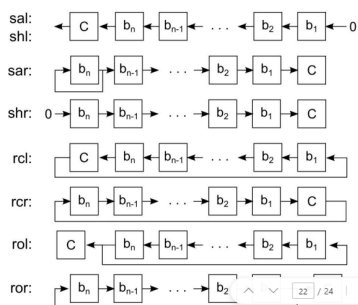
Quanto detto finora vale solo per shift di una posizione, ma per shiftare più posizioni? **Lo faccio multi livello :**



Da notare che questo circuito è molto complesso dal punto di vista funzionale e circuitale : si può migliorare?? Si , usando il **barrel shifter** : componenti e livelli ridotti . Non sfrutta la decomposizioni in reti iterative, ma usa interruttori (buffer tri state) :



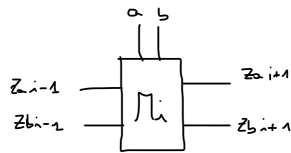
Il decoder sceglie quale canale verrà usato. Tipicamente le operazioni che può fare questo dispositivo sono le seguenti :



- Uno shifter non implementa soltanto traslazioni a destra e a sinistra
 - le operazioni viste fino ad ora prendono il nome di *shift logico*
- Altre operazioni tipiche sono:
 - rotazioni
 - shift aritmetico
- In alcuni casi, viene preso in considerazione anche un *bit carry*

Vediamo ora un esempio di comparatore : prendiamo due numeri e confrontiamoli :

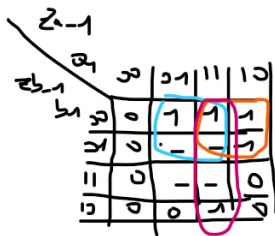
A=010011 Risultato di comparazione : **1**10000
 B=000111 **0**01100



Se i bit sono uguali come uscita avrò la propagazione dello stato precedente (tratto rosso è propagazione) . Andiamo ora a costruire la tabella di verità :

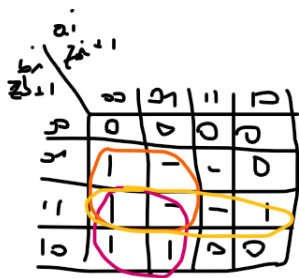
Za (i-1)	Zb(i-1)	Ai	Bi	Za(i+1)	ZB(i+1)
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	1	0
0	0	1	1	0	0
0	1	0	0	0	1
0	1	0	1	0	1
0	1	1	0	1	0
0	1	1	1	0	1
1	0	0	0	1	0
1	0	0	1	0	1
1	0	1	0	1	0
1	0	1	1	1	0

Le altre condizioni sono don't care . Minimizziamo con karnaugh :



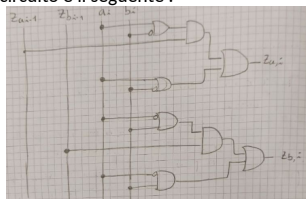
$$\underline{Z_{a,i}} = a_i Z_{a,i-1} + \bar{b}_i Z_{a,i-1} + a_i b_i = Z_{a,i-1} (a_i + \bar{b}_i) + a_i b_i$$

Per la propagazione lo stesso :



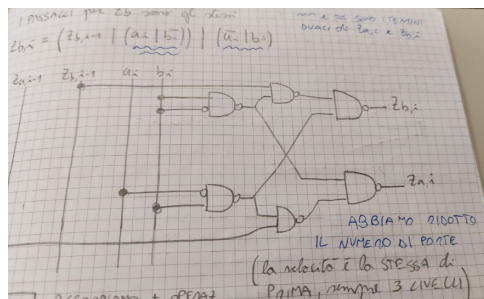
$$\underline{Z_{b,i}} = b_i Z_{b,i-1} + Z_{b,i-1} \bar{a}_i + b_i \bar{a}_i = Z_{b,i-1} (\bar{a}_i + b_i) + \bar{a}_i b_i$$

Il cui circuito è il seguente :

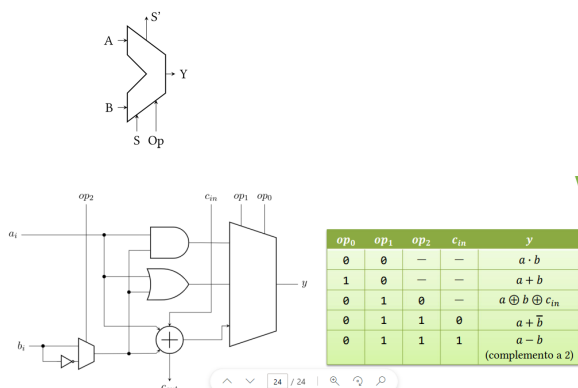


In ogni modulo ho 8 porte : possiamo semplificare questo circuito minimo?? Si in forma analitica :

$$\begin{aligned} Z_{a,i} &= Z_{a,i-1} (a_i + \bar{b}_i) + a_i \bar{b}_i = Z_{a,i-1} (a_i + \bar{b}_i) + a_i \bar{b}_i = \\ &= Z_{a,i-1} (a_i + \bar{b}_i) + a_i \bar{b}_i = Z_{a,i-1} (a_i + \bar{b}_i) + (a_i | \bar{b}_i) = \\ &= (Z_{a,i-1} | (a_i | \bar{b}_i)) | (a_i | \bar{b}_i) = \\ &= (Z_{a,i-1} | (a_i | \bar{b}_i)) | (a_i | \bar{b}_i) \end{aligned}$$

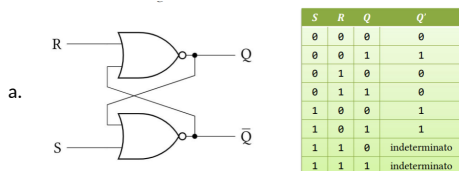


Proviamo a mettere insieme vari pezzi visti finora : andiamo a vedere un componente programmabile : la **ALU (arithmetic and logic unit)**: dispositivo che opera su parole di dimensione fissa , permette di fare molte operazioni ed ha bisogno di due segnali di ingresso (op-code : codice che determina operazione da fare) : fa diminuire i fili e la circuiteria :



Le operazioni implementate sono 5 apposto delle 8 (3 bit di controllo) . Il dispositivo che contiene gli op-code è un mux a 2 ingressi : in base alla configurazione restituisce la funzione opportuna . La terza configurazione è un sommatore : full-adder , mentre nel penultima operazione è un inverter, invece nell'ultimo caso di usa come sottrattore (somma complemento a 2). Il circuito calcola tutte le operazioni possibili contemporaneamente. I circuiti visti finora sono chiamati combinatori in quanto combinano variabili booleane per avere una funzione in uscita : ricordiamo la stabilità del circuito : non possiamo costruire nessun elemento di memoria . Come possiamo ovviare a ciò? Usiamo l'uscita come ingresso : anello di retroazione . Vediamo un esempio :

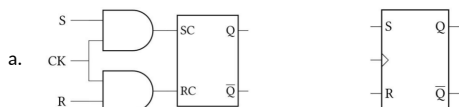
1. **Latch (questo circuito ha uno stato):** output precedente influenza input successivo . Consente di immagazzinare un'informazione se il risultato non cambia al variare del tempo.



- b. Da notare che il valore di r e s se sono soli ad uno , il latch 1-0 il latch e in modo set, analogamente per il reset (0-1).

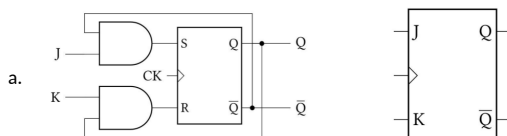
- c. **Unico stato inconsistente si ha quando entrambi gli input sono settati ad 1**

2. **Flip flop (miglioria del latch: output stabili):**



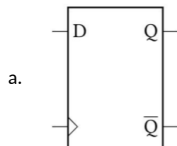
- b. Il clock serve a tenere il tempo e tiene traccia del tempo che le reti ci mettono a stabilizzarsi : quindi evita di leggere segnali instabili .
- c. Anche questo dispositivo non sopporta la configurazione 11

3. **Flip flop J-k** : segnali set / reset nominati e sfrutta seconda rete di retrazione per filtrare il segnale



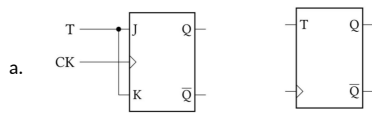
- b. **Con questo ff non è un problema la configurazione 11**

4. **Flip-flop D (scrive un solo bit)** : non mantiene stabile l'input . Ha due segnali opposti

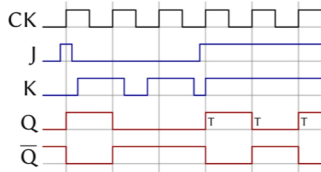


- b. D sta per delay : se 1 su delay devo aspettare un colpo di clock per avere l'uscita : introduce un ritardo di un colpo di clock per avere uscita

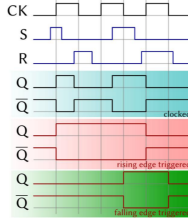
5. **Flip-flop T (switch)** : inverte il valore dell'uscita :



Come funziona il clock nel tempo? Segnale che per un certo tempo vale 0 e per un certo periodo vale 1 : andiamo a vedere il fronte di commutazione (passaggio 0-1 e viceversa): momento in cui il mio ff commuta :



Vediamo ora un esempio : come si evolve il segnale nel latch s-r:

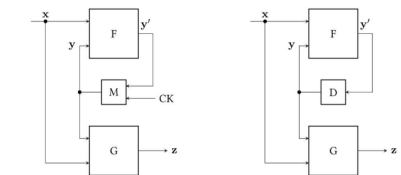


Le reti sequenziali sono una macro famiglia delle reti combinatorie viste finora . Quindi le relazioni che caratterizzano questi circuiti sono molteplici : infatti non vi è una sola equazione ma bensì due : sistema di equazioni booleane , in quanto il circuito evolve nel tempo. Quindi si parla di stati interni diversi del circuito : output viene chiamato Z che viene calcolata con la funzione booleana G (che non accetta solo X ma anche Y (vettore)), mentre vi è altra funzione F che accetta input e stato interno (vettore), la quale determinerà una transizione di stato (F calcola funzione in cui si troverà il nostro circuito): vediamo un esempio il latch S-R:

S	R	Q	Q'
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	indeterminato
1	1	1	indeterminato

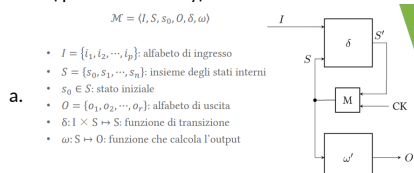
$$\begin{cases} y' = f(x, y) \\ z = g(x, y) \end{cases}$$

Ci sono due categorie di reti sequenziali : **sincrone ed asincrone** . Nelle prime si calcola la transizione in determinati istanti di tempo ben precisi e controllabili dall'esterno (clock) , mentre nelle seconde non si ha il controllo delle transizioni : circuiti analogici .



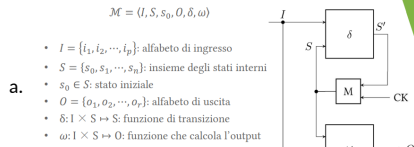
F e G sono blocchi logici che implementano funzioni logiche , attraverso circuiteria . Nel primo caso (sx) servono degli elementi per mantenere le info quindi devo usare dei ff (visto che il latch non supporta 1-1) . **Per calcolare il tempo di campionamento devo aspettare almeno il ritardo di propagazione di F**. Per quanto riguarda invece il modulo M come faccio a garantire che non vi sia un valore sballato? Uso il fronte di commutazione per attivare lettura / scrittura : magari scrittura su fronte discesa, mentre lettura in salita . Mentre nel secondo esempio il blocco D (delay) , non ho intervalli ben precisi per manipolare informazioni: dopo che il processamento è finito e ritardo del blocco D , il segnale sarà scritto/letto (asincrono). Andiamo ora a definire le reti F e G: andiamo a definire le **Macchine (automi) a stati finiti** : modello matematico che descrive l'evoluzione del sistema nel tempo . E' un modello astratto che mostra in un dato istante il sistema si trova in uno stato ben preciso , ed attraverso gli **eventi** (condizione che in un dato istante fa succedere qualcosa) effettua una **transizione di stato**. **Le transizioni di stato vengono determinate dal blocco F** . Anche con questi modelli si possono determinare degli output. Due varianti principali : **Mealy E Moore** : differiscono nel modo di generare output : nel primo output dipende dallo stato e dalla transizione innescata , mentre nel secondo dipende unicamente da output. Vediamoli nel dettaglio :

1. Moore (più stati di Mealy)



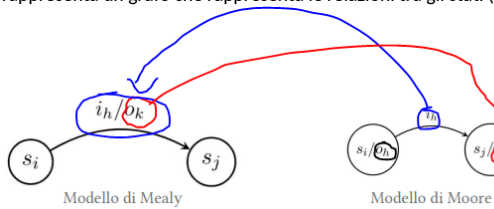
- b. La funzione delta serve per calcolare il blocco F : dato stato ed input , permette di calcolare funzione "prossimo stato".
c. La funzione "omega" permette di calcolare l'output .

2. Mealy (uno stato più transizioni) :



- b. Cambia "omega" necessita di stato attuale e di input : caso più generale di ASF.

Vediamo ora delle rappresentazione per questo formalismo : o **diagramma a stati** o **tabella degli stati** : il primo rappresenta un grafo che rappresenta le relazioni tra gli stati (cerchietti) e transizioni :



Mentre per la seconda associa degli stati a delle transizioni in funzione dell'input e determina output : riga sono gli stati , mentre sulla prima colonna gli input :

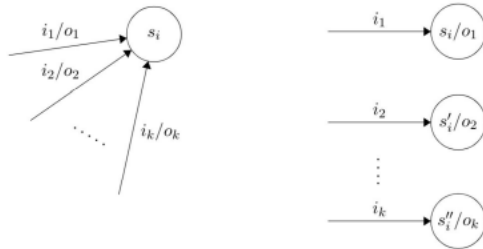
	i_1	i_2	...	i_j	...	i_p	
s_1							
s_2							
$\vdots$							
s_i				$\delta(i_i, s_i) / \omega(i_i, s_i)$			
$\vdots$							
s_n							

Modello di Mealy

	i_1	i_2	...	i_j	...	i_p	ω'
s_1							
s_2							
$\vdots$							
s_i				$\delta(i_i, s_i)$			$\omega'(i_i, s_i)$
$\vdots$							
s_n							

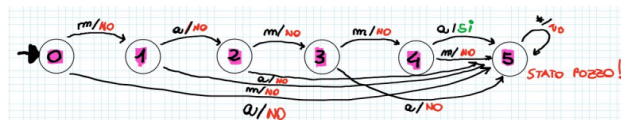
Modello di Moore

Possibile passare da Mealy a Moore e viceversa in quanto sono equivalenti .



Vediamo ora la sintesi : cominciamo dal blocco M (memoria) : lo creiamo con una serie di ff di tipo D (il numero dipende dal vettore Y), per quanto riguarda la funzione "delta" è necessario realizzare un circuito di commutazione per aggiornare lo stato di tutti i ff (**equazione di eccitazione di un ff**) , mentre per la rete "omega" uso un circuito di commutazione per generare ciascun bit di output .
Vediamo un esempio : Stringa in input "MAMMA" :

Input = { m,a } Output = { si , no }



In ogni stato si ha il "resoconto" di quanto fatto finora (memorizza sequenza input finora) . Da notare il "5" sullo stato 5 : qualunque carattere dopo la stringa "MAMMA" non è valido!! . Andiamo a vedere ora la tabella delle transizioni :

	m	a
0	1/no	5/no
1	5/no	2/no
2	3/no	5/no
3	4/no	5/si
4	5/no	5/si
5	5/no	5/no

Se sto nello stato 0 e leggo m, vado nello stato 1 con output no.

1/no

II II

delta omega

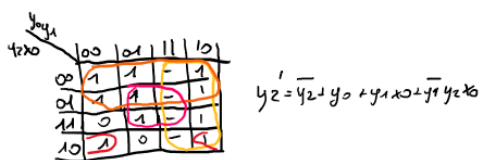
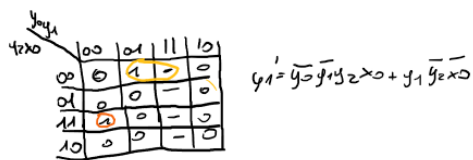
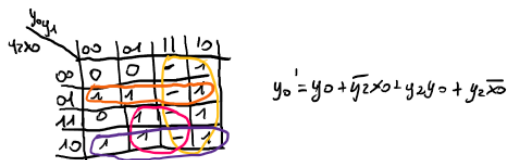
I	X
m	0
a	1

O	Z
Si	1
No	0

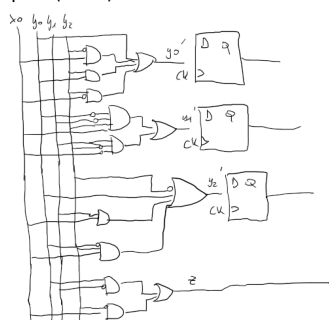
A volta gli input e gli output, sono già delle codifiche. Li non ho posso scegliere.

Metto tutto insieme : uso il binomiale per vedere quante variabili necessitano : ne servono 6 : in quanto binomiale $(4\ 2) = (4*3)/2$.

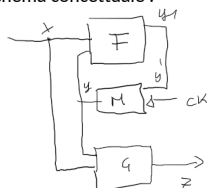
INPUT				OUTPUT			
y_3	y_2	y_1	y_0	y_3'	y_2'	y_1'	z
0	0	0	0	0	0	1	0
0	0	0	1	1	0	1	0
0	0	1	0	0	1	1	0
0	0	1	1	1	0	0	0
0	1	0	0	1	1	1	0
0	1	0	1	1	1	0	0
0	1	1	0	1	0	1	0
0	1	1	1	1	0	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	0	1	0
1	0	1	1	0	0	0	0
1	1	0	0	0	0	1	1
1	1	0	1	0	0	0	1
1	1	1	0	0	1	1	1
1	1	1	1	0	1	0	1



Mentre per z (uscita) visto che è unico mintermine , prendo la codifica :



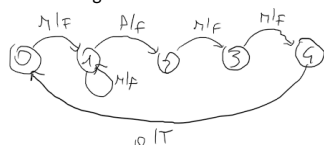
Con schema concettuale :



Vediamo ora l'automa senza stato pozzo (stato 5) sequenza di 5 caratteri sia mamma:



Mentre se voglio che le ultime 5 lettere siano mamma :



Stato 5 e stato 0 sono lo stesso stato